

10강

머신러닝

로지스틱 회귀

통계·데이터과학과 박준희



✧ 학습목차

- 1 분류 문제의 접근 방법
- 2 로지스틱 회귀
- 3 Basis 확장 및 뉴턴 방법



1

분류 문제의 접근 방법



1

분류 문제의 접근 방법

● 분류 문제와 로지스틱 회귀

이진 분류(binary classification)는 입력 벡터 $\mathbf{x}$ 가 주어졌을 때, 클래스 C_1 혹은 클래스 C_0 에 속할 확률을 예측하는 문제이다.

로지스틱 회귀는 **시그모이드(Sigmoid)** 함수를 통해

$$p(C_1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + w_0)$$

형태로 확률을 계산하는 **Discriminative model**의 대표적 예이다.

1

분류 문제의 접근 방법

● 분류 문제와 로지스틱 회귀

- 생성형 접근 (Generative Approach)

- 데이터가 생성되는 과정 자체를 모델링
- $p(x, y)$ 를 직접 추정, $p(y | x) = \frac{p(x, y)}{p(x)}$ 이용하여 분류/예측

- 식별형 접근 (Discriminative Approach)

- 주어진 입력 x 가 어떤 레이블 y 로 식별되는지를 모델링
- $p(y | x)$ 또는 결정 경계(Decision boundary)를 직접 학습하여 분류/예측

1

분류 문제의 접근 방법

● 생성형 접근에서의 유도

$$p(x | C_1) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_1, \Sigma), \quad p(x | C_0) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_0, \Sigma)$$

위와 같이 **같은 공분산 행렬** Σ 를 사용하고, **평균만 다른** 두 개의 다변량 정규분포를 가정해보자. 두 클래스의 사전확률을 각각 $p(C_1), p(C_0)$ 라 할 때,

$$p(C_1 | \mathbf{x}) = \frac{p(\mathbf{x} | C_1) p(C_1)}{p(\mathbf{x} | C_1) p(C_1) + p(\mathbf{x} | C_0) p(C_0)} = \frac{1}{1 + \frac{p(\mathbf{x}|C_0) p(C_0)}{p(\mathbf{x}|C_1) p(C_1)}}$$

로그를 취하거나, 가우시안 형태를 풀어서 쓰면 다음과 같은 지수함수 형태가 된다.

$$p(C_1 | \mathbf{x}) = \frac{1}{1 + \exp(-(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \Sigma^{-1} \mathbf{x} + \text{const.})}.$$

즉, **결과적으로 시그모이드 형태**로 나타남을 확인할 수 있다.

1

분류 문제의 접근 방법

● 식별형 접근 (로지스틱 회귀)

로지스틱 회귀에서 사용하는 로지스틱 시그모이드 함수 $\sigma(a)$ 는

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

로 정의되며, 다음과 같은 성질을 갖는다.

1. $\sigma(-a) = 1 - \sigma(a)$
2. $\sigma'(a) = \sigma(a) \sigma(-a) = \sigma(a) (1 - \sigma(a))$

1

분류 문제의 접근 방법

● 식별형 접근 (로지스틱 회귀)

시그모이드 함수 $\sigma(a)$ 의 역함수는 로짓(logit)함수로 다음과 같다.

$$a = \log \left(\frac{\sigma}{1 - \sigma} \right)$$

로지스틱 회귀에서는

$$p(C_1 | \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})} = \sigma(\mathbf{w}^T \mathbf{x}).$$

1

분류 문제의 접근 방법

● 식별형 접근(로지스틱 회귀)

오즈비(odds ratio)와 로그 오즈(log-odds)는 다음과 같이 정의된다.

$$\frac{p(C_1 | \mathbf{x})}{p(C_0 | \mathbf{x})} = \exp(\mathbf{w}^T \mathbf{x}), \quad \log \left(\frac{p(C_1 | \mathbf{x})}{p(C_0 | \mathbf{x})} \right) = \mathbf{w}^T \mathbf{x}.$$

결론적으로, log를 취했을 때 선형식이 된다는 점이 로지스틱 회귀의 큰 특징이다.

2

로지스틱 회귀



2 로지스틱 회귀

● Maximum Likelihood Estimation

훈련 데이터 $\{(\mathbf{x}_n, t_n)\}$ ($t_n \in \{0, 1\}$)에 대해 로지스틱 회귀의 파라미터 $\mathbf{w}$ 를 학습하려면, Log-Likelihood(우도)를 최대화한다. 각 샘플에 대해

$$p(t_n | \mathbf{x}_n) = \sigma(\mathbf{w}^T \mathbf{x}_n)^{t_n} [1 - \sigma(\mathbf{w}^T \mathbf{x}_n)]^{1-t_n}.$$

우도의 로그를 취하면 **Cross-Entropy** 형태가 된다.

$$\ell(\mathbf{w}) = \sum_{n=1}^N \left[t_n \log \sigma(\mathbf{w}^T \mathbf{x}_n) + (1 - t_n) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}_n)) \right].$$

이를 최대화하는 것은 곧 로지스틱 손실(Logistic loss, Binary cross-entropy)를 최소화하는 것과 동일하다.

2 로지스틱 회귀

● Maximum Likelihood Estimation

- $\mathbf{w}^T \mathbf{x}_n \equiv a_n, \sigma(a_n) \equiv y_n$ 라고 하면,

$$\ell(\mathbf{w}) = \sum_{n=1}^N [t_n \log y_n + (1 - t_n) \log(1 - y_n)].$$

$$\sigma'(a) = \sigma(a) \sigma(-a) = \sigma(a) (1 - \sigma(a))$$

$$\nabla_{\mathbf{w}} \log y_n = \frac{1}{y_n} \nabla_{\mathbf{w}}(y_n) = \frac{1}{y_n} \times y_n(1 - y_n) \mathbf{x}_n = (1 - y_n) \mathbf{x}_n.$$

$$\nabla_{\mathbf{w}} \log(1 - y_n) = \frac{-\nabla_{\mathbf{w}}(y_n)}{1 - y_n} = -y_n \mathbf{x}_n.$$

2 로지스틱 회귀

● Maximum Likelihood Estimation

$$\nabla \ell(\mathbf{w}) = \sum_{n=1}^N \left[t_n - \sigma(\mathbf{w}^T \mathbf{x}_n) \right] \mathbf{x}_n.$$

이를 이용해 경사상승(gradient ascent) 혹은 경사하강(손실 관점) 방법으로 업데이트할 수 있다.

2 로지스틱 회귀

● 로지스틱 회귀 예제

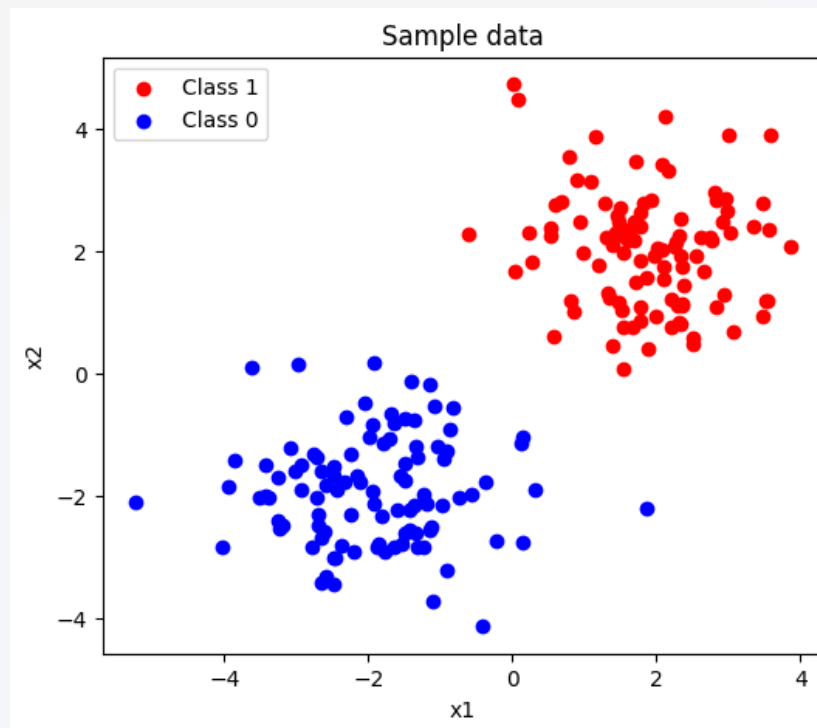
```
# 클래스 1 (label=1) 샘플들: 대략 (2,2) 근방에 모여있게 만든다.  
N = 100 # 각 클래스당 샘플 수  
x1_class1 = np.random.normal(loc=2.0, scale=1.0, size=N)  
x2_class1 = np.random.normal(loc=2.0, scale=1.0, size=N)  
X_class1 = np.column_stack((x1_class1, x2_class1))  
y_class1 = np.ones(N)  
  
# 클래스 0 (label=0) 샘플들: 대략 (-2,-2) 근방에 모여있게 만든다.  
x1_class0 = np.random.normal(loc=-2.0, scale=1.0, size=N)  
x2_class0 = np.random.normal(loc=-2.0, scale=1.0, size=N)  
X_class0 = np.column_stack((x1_class0, x2_class0))  
y_class0 = np.zeros(N)  
  
# 전체 데이터 합치기  
X = np.vstack((X_class1, X_class0)) # shape = (2N, 2)  
t = np.concatenate((y_class1, y_class0)) # shape = (2N, )
```

출처:파이썬 프로그램 실습내용 중

2 로지스틱 회귀

● 로지스틱 회귀 예제

```
# 시각화
plt.figure(figsize=(6,5))
plt.scatter(X_class1[:,0], X_class1[:,1], color='r', label='Class 1')
plt.scatter(X_class0[:,0], X_class0[:,1], color='b', label='Class 0')
plt.xlabel('x1')
plt.ylabel('x2')
plt.legend()
plt.title('Sample data')
plt.show()
```



출처:파이썬 프로그램 실습내용 중

2 로지스틱 회귀

● 로지스틱 회귀 예제

```
# -----
# 2. 시그모이드 함수 구현
# -----
def sigmoid(a):
    return 1.0 / (1.0 + np.exp(-a))

# -----
# 3. 로지스틱 회귀 (경사하강법) 구현
# -----

# (1) 파라미터 초기화
#     feature dimension = 2, 따라서 w는 (3,) shape (bias 포함 위해 +1)
w = np.random.randn(3) * 0.01

# (2) 학습을 위해 X에 bias 항 추가
#     X.shape = (2N, 2)
#     --> X_with_bias.shape = (2N, 3) [마지막 col에 1]
X_with_bias = np.hstack([X, np.ones((X.shape[0], 1))])
```

출처:파이썬 프로그램 실습내용 중

2 로지스틱 회귀

● 로지스틱 회귀 예제

```
# (3) 손실함수(이진크로스엔트로피) & 경사 계산 함수
def compute_loss_and_grad(w, Xb, t):
    """
    w: (3,) vector / Xb: (2N, 3) / t: (2N, )
    """

    #  $z = w^T x$ 
    z = Xb.dot(w) # shape=(2N,)
    #  $y = \text{sigmoid}(z)$ 
    y = sigmoid(z) # shape=(2N,)

    # (a) loss (cross-entropy)
    #  $-\sum [t_n \log y_n + (1 - t_n) \log(1 - y_n)]$ 
    eps = 1e-10
    loss = - np.sum( t*np.log(y+eps) + (1-t)*np.log(1-y+eps) )

    # (b) gradient
    #  $\nabla l = \sum (y_n - t_n) * x_n$ 
    grad = Xb.T.dot(y - t) # shape=(3,)

    return loss, grad
```

출처:파이썬 프로그램 실습내용 중

2 로지스틱 회귀

● 로지스틱 회귀 예제

```
# (4) 학습 루프
```

```
learning_rate = 0.001
```

```
n_epochs = 30
```

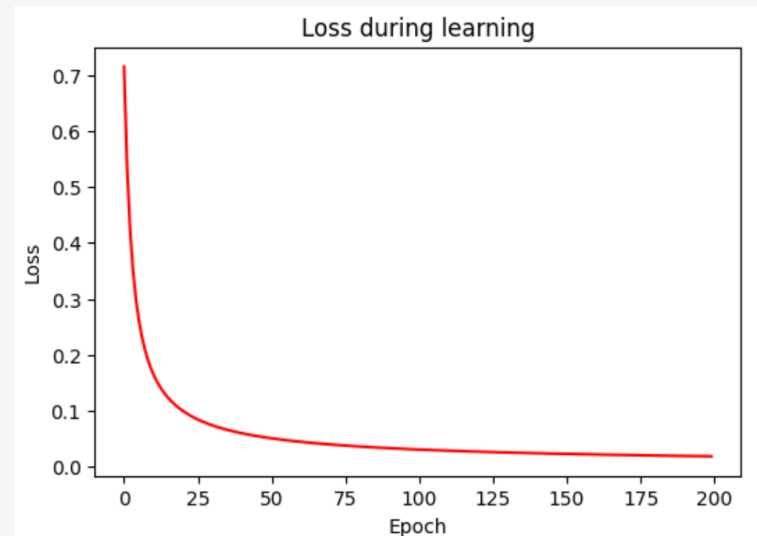
```
loss_history = []
```

```
for epoch in range(n_epochs):
```

```
    loss, grad = compute_loss_and_grad(w, X_with_bias, t)
```

```
    w = w - learning_rate * grad # 경사하강 업데이트
```

```
    loss_history.append(loss)
```



2 로지스틱 회귀

● 로지스틱 회귀 예제

```
# -----
# 4. Decision Boundary 시각화
# -----

plt.figure(figsize=(6,5))
plt.scatter(X_class1[:,0], X_class1[:,1], color='r', label='Class 1')
plt.scatter(X_class0[:,0], X_class0[:,1], color='b', label='Class 0')

# 결정 경계를 위해 x1, x2 범위에서 그리드 생성
x1_vals = np.linspace(-5, 5, 100)
x2_vals = np.linspace(-5, 5, 100)
X1, X2 = np.meshgrid(x1_vals, x2_vals)

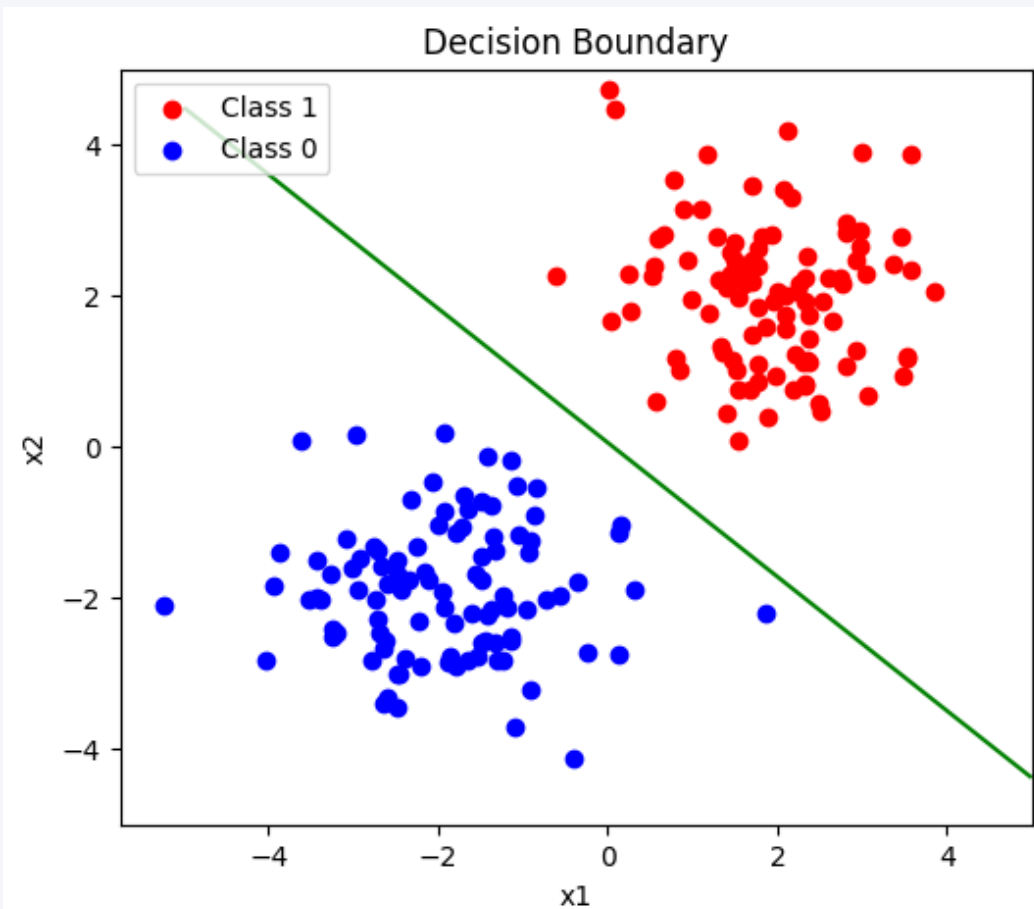
Xb_grid = np.c_[X1.ravel(), X2.ravel(), np.ones_like(X1.ravel())]
y = sigmoid(Xb_grid.dot(w))
y = y.reshape(X1.shape)

# contour(등고선)로 p=0.5인 라인만 그려주기
plt.contour(X1, X2, y, levels=[0.5], colors='g')
```

출처:파이썬 프로그램 실습내용 중

2 로지스틱 회귀

● 로지스틱 회귀 예제



출처:파이썬 프로그램 실습내용 중

3

Basis 확장 및 뉴턴 방법



3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

입력 $\mathbf{x}$ 가 주어졌을 때, 클래스 C_1 에 속할 확률을

$$y_n = p(C_1 \mid \mathbf{x}_n) = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n)), \quad \text{where } \sigma(a) = \frac{1}{1 + e^{-a}}$$

로 정의해보자. 여기서

- $\phi(\mathbf{x}_n)$ 은 feature transformation 역할을 하는 기저 함수(basis function)이다.
- y_n 은 $\mathbf{x}_n$ 가 주어졌을 때, 클래스 C_1 일 확률 예측치이다.

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

훈련 데이터가

$$D = \{(\mathbf{x}_n, t_n)\}_{n=1}^N, \quad t_n \in \{0, 1\}$$

라고 할 때, 로지스틱 모델에서 각 표본 $(\mathbf{x}_n, t_n)$ 에 대한 조건부 확률은

$$p(t_n | y_n) = y_n^{t_n} (1 - y_n)^{1-t_n}.$$

즉, 만약 $t_n = 1$ 이면 $p(t_n | y_n) = y_n$,

만약 $t_n = 0$ 이면 $p(t_n | y_n) = 1 - y_n$.

따라서 전체 데이터셋에 대한 우도(likelihood)는

$$L(\mathbf{w}) = p(\mathbf{t} | \mathbf{w}) = \prod_{n=1}^N p(t_n | y_n) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n},$$

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

$$\begin{aligned}
 \ell(\mathbf{w}) &= \log L(\mathbf{w}) \\
 &= \log \prod_{n=1}^N \left(y_n^{t_n} (1 - y_n)^{1-t_n} \right) \\
 &= \sum_{n=1}^N \left(t_n \log y_n + (1 - t_n) \log(1 - y_n) \right).
 \end{aligned}$$

여기서 $y_n = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))$.

로그 우도를 **최대화**하는 대신, 이를 음수 부호로 바꾼 negative log-likelihood를 **최소화**하는 문제로 바꿔서 푼다:

$$\begin{aligned}
 E(\mathbf{w}) &= -\log L(\mathbf{w}) \\
 &= -\ell(\mathbf{w}) \\
 &= -\sum_{n=1}^N \left[t_n \log y_n + (1 - t_n) \log(1 - y_n) \right].
 \end{aligned}$$

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

앞에서처럼 미분하여 전개해 보면,

$$\nabla_{\mathbf{w}} E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi(\mathbf{x}_n).$$

- $y_n = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))$
- 따라서 경사하강법(gradient descent) 업데이트 식은 다음과 같다.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \sum_{n=1}^N (y_n - t_n) \phi(\mathbf{x}_n),$$

여기서, η 는 학습률(learning rate)이다.

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

- 뉴턴방법(Newton's method)

- 일반적인 로지스틱 회귀에서 경사하강법(Gradient Descent)을 쓸 수도 있지만,
- **Newton's method**는 2차 미분(헤시안, Hessian) 정보를 활용하여 더 빠르게 수렴할 수 있는 장점이 있다.

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

- 뉴턴방법(Newton's method)

그래디언트는 1차 미분 벡터로,

$$\nabla_{\mathbf{w}} E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n = \Phi^T (\mathbf{y} - \mathbf{t}).$$

여기서, Φ 는 n 행이 ϕ_n^T 인 $N \times M$ 디자인 매트릭스이다.

헤시안(Hessian) $\mathbf{H}$ 은 2차 미분 행렬로,

$$\mathbf{H} = \nabla_{\mathbf{w}} \nabla_{\mathbf{w}} E(\mathbf{w}) = \sum_{n=1}^N y_n (1 - y_n) \phi_n \phi_n^T = \Phi^T R \Phi.$$

여기서, R 은 n 번째 대각원소가 $y_n(1 - y_n)$ 인 대각행렬이다.

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀: Basis expansion

■ 뉴턴방법(Newton's method)

Newton's method에서는 최적화하고자 하는 함수 $E(\mathbf{w})$ 에 대해,

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \mathbf{H}^{-1} \nabla_{\mathbf{w}} E(\mathbf{w}_{\text{old}}).$$

즉, 다음 식

$$\begin{aligned} \mathbf{w}_{\text{new}} &= \mathbf{w}_{\text{old}} - (\Phi^T R \Phi)^{-1} \Phi^T (\mathbf{y} - \mathbf{t}) \\ &= (\Phi^T R \Phi)^{-1} (\Phi^T R \Phi \mathbf{w}_{\text{old}} - \Phi^T (\mathbf{y} - \mathbf{t})) \\ &= (\Phi^T R \Phi)^{-1} \Phi^T R \mathbf{z} \end{aligned}$$

을 기준으로 업데이트한다 ($\mathbf{z} = \Phi \mathbf{w}_{\text{old}} - R^{-1}(\mathbf{y} - \mathbf{t})$). 위 식을 **Iteratively Reweighted Least Squares, IRLS**라고 부른다.

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
def phi_polynomial_2nd(X):  
    """  
    X: shape (N, 2) -> 예: (x1, x2)  
    return: shape (N, 6) -> [1, x1, x2, x1^2, x2^2, x1*x2]  
    """  
  
    x1 = X[:,0]  
    x2 = X[:,1]  
  
    # 1, x1, x2, x1^2, x2^2, x1*x2  
    phi_list = [  
        np.ones_like(x1),  
        x1,  
        x2,  
        x1**2,  
        x2**2,  
        x1*x2  
    ]  
  
    return np.column_stack(phi_list)
```

출처:파이썬 프로그램 실습내용 중

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
# 예시 입력
X_sample = np.array([[1.0, 2.0],
                     [2.5, -1.0],
                     [-3.0, 0.5]])

phiX = phi_polynomial_2nd(X_sample)
print(phiX)
# shape: (3, 6)
```

```
[[ 1.    1.    2.    1.    4.    2. ]
 [ 1.    2.5  -1.    6.25  1.   -2.5 ]
 [ 1.   -3.    0.5    9.    0.25 -1.5 ]]
```

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

[6]: # 위의 주어진 데이터를 이용하여 Phi 행렬 생성

```
Phi = phi_polynomial_2nd(X)
print("Phi.shape =", Phi.shape)
```

```
Phi.shape = (200, 6)
```

```
def negative_log_likelihood(w, Phi, t):
```

```
    """
```

```
    w: (6,) - 2차 다항 확장 후의 차원
```

```
    Phi: (2N, 6)
```

```
    t: (2N,)
```

```
    """
```

```
    z = Phi.dot(w) # (2N,)
```

```
    y = sigmoid(z) # (2N,)
```

```
    eps = 1e-12
```

```
    nll = -np.sum(t*np.log(y+eps) + (1-t)*np.log(1-y+eps))
```

```
    return nll
```

출처:파이썬 프로그램 실습내용 중

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
def gradient(w, Phi, t):  
    z = Phi.dot(w)    # (2N,)   
    y = sigmoid(z)    # (2N,)   
    grad = Phi.T.dot(y - t) # shape=(6,)   
    return grad  
  
def hessian(w, Phi, t):  
    z = Phi.dot(w)    # (2N,)   
    y = sigmoid(z)    # (2N,)   
    r = y*(1-y)      # (2N,) 대각 성분   
    R = np.diag(r)    # (2N, 2N)   
    H = Phi.T.dot(R).dot(Phi) # (6, 6)   
    return H
```

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
# 초기 파라미터 ( $\theta$ 로 초기화)  
w = np.zeros(Phi.shape[1])  
max_iter = 50  
tol = 1e-6  
  
loss_history = []
```

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
for i in range(max_iter):
    grad = gradient(w, Phi, t)
    H = hessian(w, Phi, t)
    delta = np.linalg.inv(H).dot(grad)  #  $H^{-1} * grad$ 

    w_new = w - delta
    loss = negative_log_likelihood(w_new, Phi, t)
    loss_history.append(loss)

# 수렴 체크
if np.linalg.norm(w_new - w) < tol:
    w = w_new
    print(f"Iteration {i}: Converged.")
    break

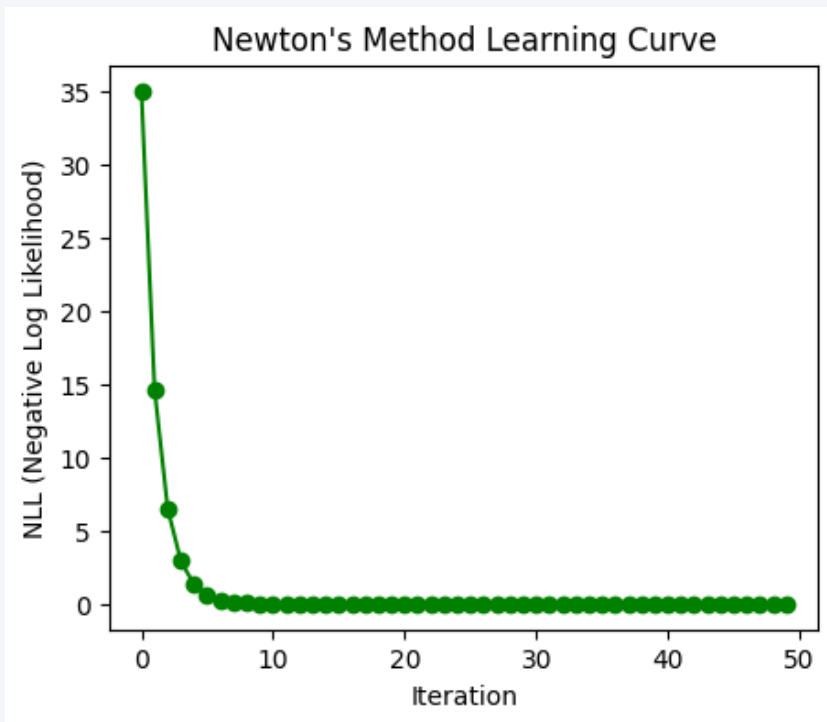
w = w_new
print(f"Iteration {i}: loss={loss:.4f}, w={w}")
```

출처:파이썬 프로그램 실습내용 중

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

- Loss는 계속 줄어드나 w 는 수렴하지 않고 계속 커지는 양상
- 완전 분리(perfect separation)문제 - 정규화등으로 해결 가능



출처:파이썬 프로그램 실습내용 중

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion

```
# (1) 2D grid 생성
x1_vals = np.linspace(-6,6,150)
x2_vals = np.linspace(-6,6,150)
X1, X2 = np.meshgrid(x1_vals, x2_vals)
grid_points = np.c_[X1.ravel(), X2.ravel()] # shape=(22500,2)

# (2) Phi 변환
Phi_grid = phi_polynomial_2nd(grid_points) # shape=(22500,6)

# (3) 예측 확률
z_grid = Phi_grid.dot(w)
y_grid = sigmoid(z_grid).reshape(X1.shape)

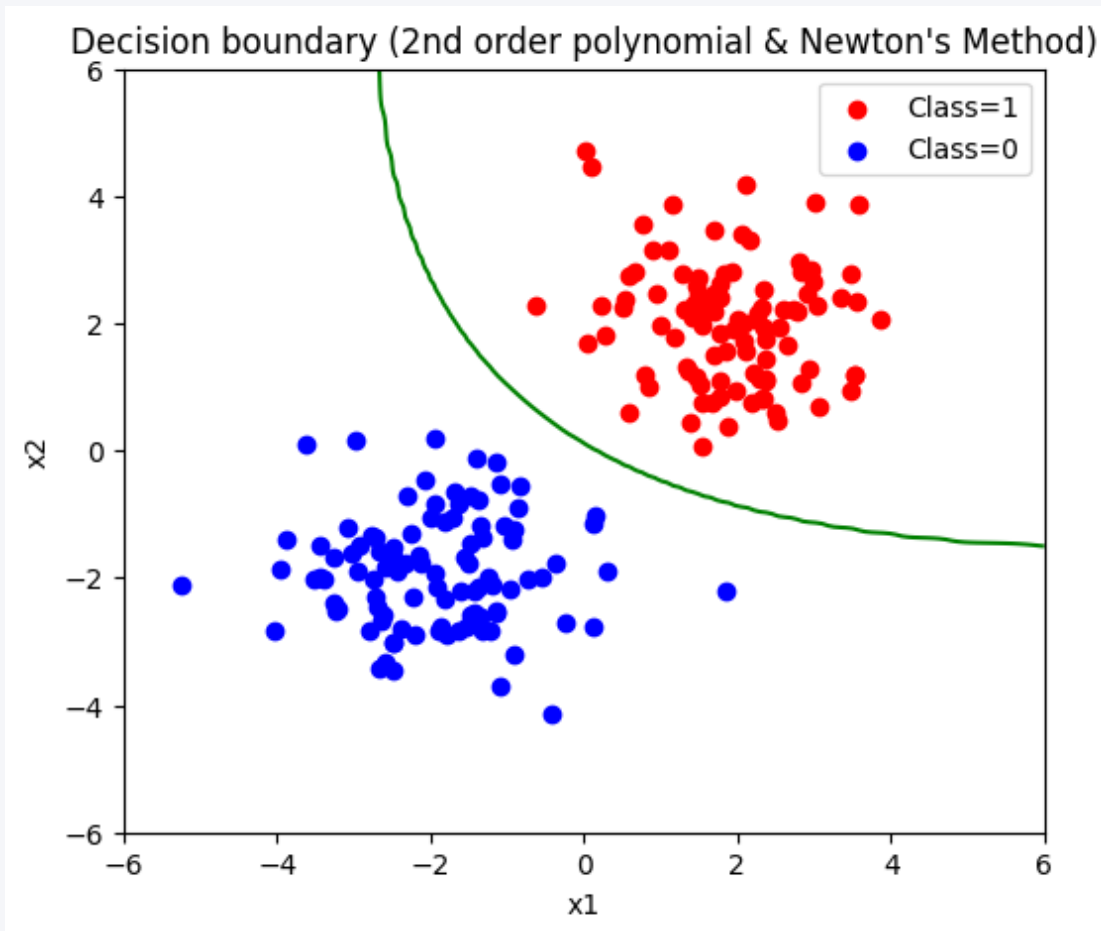
# (4) 시각화
plt.figure(figsize=(6,5))
plt.scatter(X_class1[:,0], X_class1[:,1], c='r', label='Class=1')
plt.scatter(X_class0[:,0], X_class0[:,1], c='b', label='Class=0')

# contour로 p=0.5인 라인을 그림 => 결정경계
plt.contour(X1, X2, y_grid, levels=[0.5], colors='g')
```

출처:파이썬 프로그램 실습내용 중

3 Basis 확장 및 뉴턴 방법

● 로지스틱 회귀 예제: Basis expansion



출처:파이썬 프로그램 실습내용 중

★ 정리하기

■ 분류 문제의 접근 방법

생성형 접근: 데이터가 생성되는 과정 자체를 모델링

식별형 접근: 주어진 입력이 어떤 결과로 식별되는지를 모델링

■ 로지스틱 회귀

시그모이드 함수를 이용한 선형 회귀 모형의 확장

경사하강법: 그래디언트를 이용한 최적화 방법

■ Basis 확장 및 뉴턴 방법

기저 함수를 적용하면 비선형 데이터의 이진 분류 가능

뉴턴 방법: 헤시안까지 이용한 최적화 방법

11 강

다음시간안내

서포트 벡터 분류

